

Parallel Programming HW4 Report

資工四 B10705009 邱一新

1. Implementation

The serial version's for-loop task was divided among multiple threads running in parallel on the GPU. Each thread tests different nonces (e.g., `global_tid`, `global_tid + thread_count`, ...). Once any thread finds the target nonce, it marks the result in global memory, allowing all other threads to detect that the answer has been found and terminate their work. As a result, the entire kernel exits early.

2. Parallelization and Optimization Techniques

2.1 Parallelization

I used `cudaMalloc` to allocate an `unsigned int` flag in global memory to indicate whether the nonce has been found, initializing it to zero so that all threads can track the current search status. I declared `g_block_template` and `g_target_hex[32]` in `__constant__` memory and used `cudaMemcpyToSymbol` to copy the block template and target difficulty into constant memory, allowing threads to access them efficiently.

In the GPU implementation, I rewrote all SHA-256-related functions required for nonce computation with the `__device__` qualifier, providing GPU-specific versions. The detailed optimization methods are described in the next section.

2.2 Optimization

In the original SHA-256 implementation, the function performs three `sha256_transform` operations and is designed to handle variable message lengths. However, in the mining program, only two fixed input lengths—80 and 32 bytes—are actually used. This allows for specialization by removing unnecessary length checks and reducing instruction overhead.

Additionally, the original `sha256_transform` declared an array of 64 words, which could not fit into registers and was therefore placed in slower local memory. To address this, I replaced it with a circular buffer of 16 words, reducing memory usage so it can fit into registers. I also unrolled loops and used scalar variables directly to eliminate array index computations.

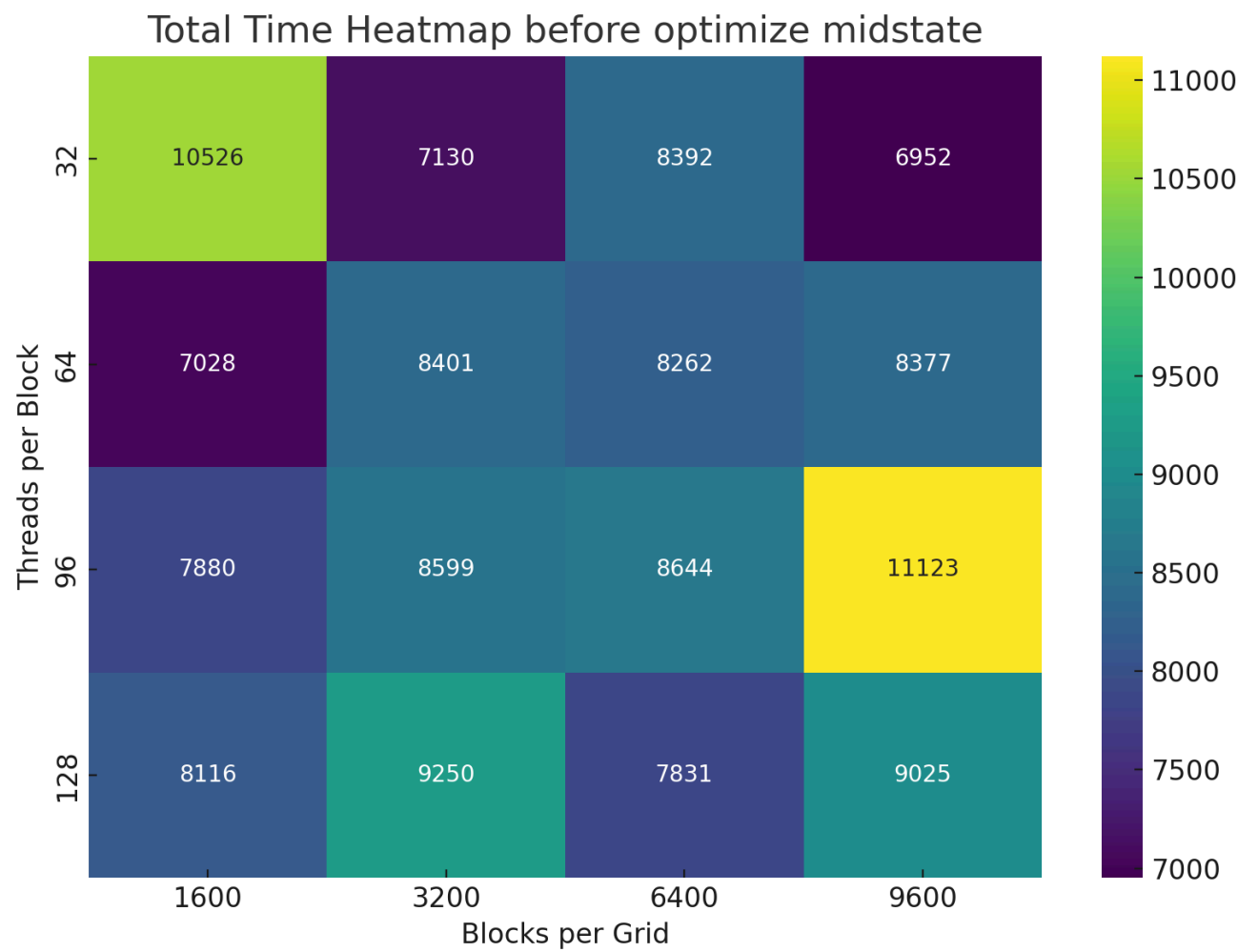
Finally, since part of the hash input (such as transaction data, timestamp, and other metadata) remains constant while only the nonce changes, I precomputed the fixed portion of the hash on the CPU—this is known as computing the *midstate*. Each thread then continues hashing from this midstate rather than recomputing the entire block

header, effectively skipping one `sha256_transform` per attempt and resulting in a substantial overall performance improvement.

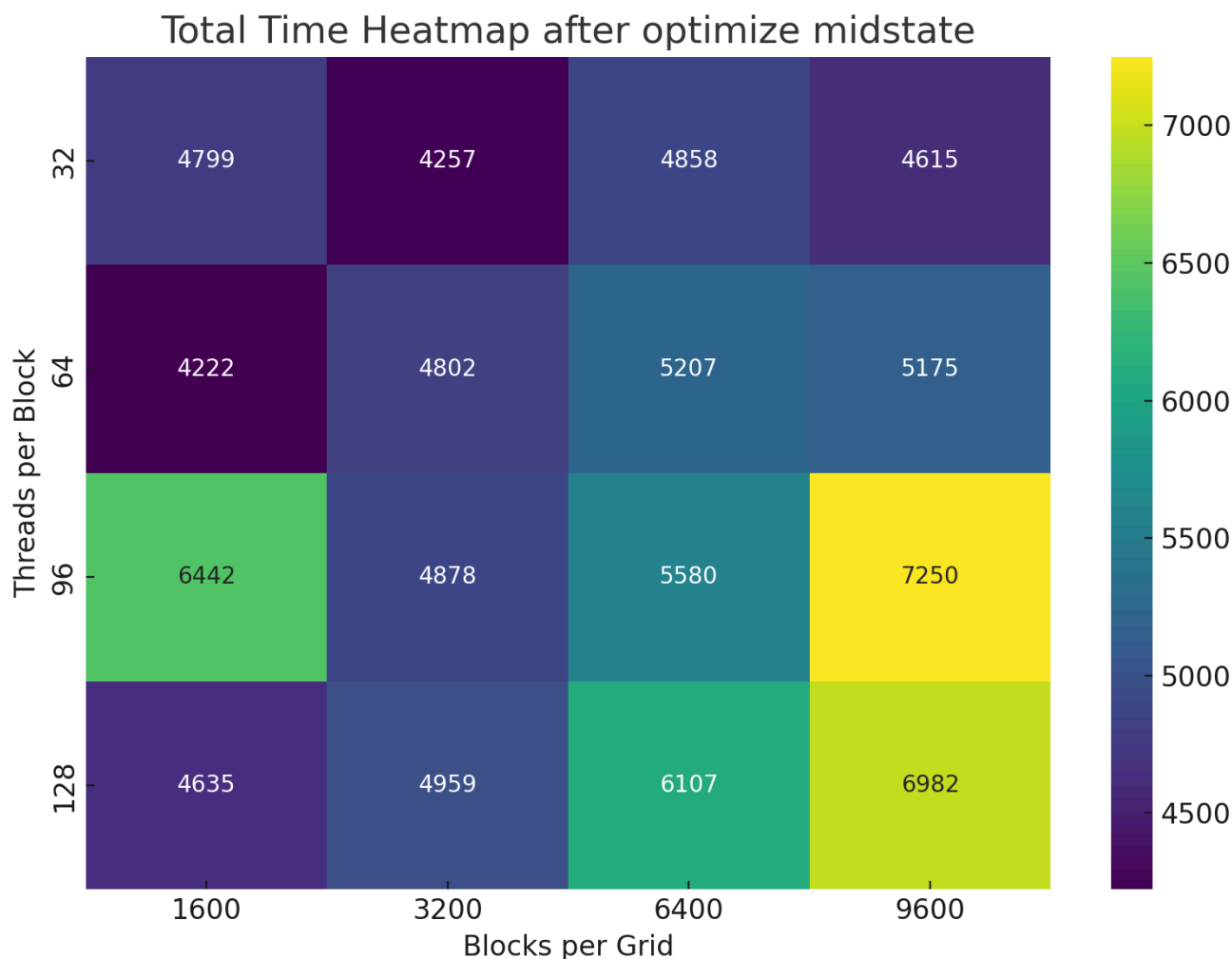
3. Experiments

3.1 Total Execution Time

I tested 16 different thread–block configurations before and after applying the midstate optimization, using the four public test cases.



totalTimeBefore.png



totalTimeAfter.png

My finding

- After the optimization, overall efficiency improved significantly, confirming that precomputing the invariant portion of the hash successfully reduced computational load.
- When observing different thread–block configurations, cases with smaller numbers of threads per block tended to perform better overall. This suggests that reducing task granularity helps the SM scheduler hide latency more effectively, thereby improving performance. As the number of blocks per grid increased, execution time showed a slight upward trend—likely because the workload was already compute-bound, so launching more blocks did not further hide latency but instead added extra launch overhead.

3.2 Execution Of Each Test Case

For each configuration, I noticed that the execution times across test cases were quite irregular and varied significantly. I believe this is because my program terminates early as soon as a valid nonce is found, instead of exhaustively checking all possibilities. As a result, the starting points and step sizes of the threads differ between configurations,

which introduces a degree of randomness—if the search happens to start near the correct nonce, it finishes much earlier purely by chance.



testcaseTime.png

4. Advanced CUDA Skills

In this assignment, I did not use advanced CUDA techniques such as streaming, page-locked memory, or asynchronous memory copy. The reason is that after parallelization, the overall computation time on the CPU became much shorter compared to the GPU, leaving little room for further performance gains from overlapping CPU–GPU pipelines. Therefore, these asynchronous techniques were not applied.